UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI



Application of Cryptography in Electronic Banking Transaction Security

Full Name	Student ID	Major
Dinh Tien Luan	23BA14180	CS
Nguyen Duc Manh	23BA14188	CS
Luong Thi Phuong	23BA14235	CS
Vu Nhat Cuong	23BA14041	CS
Nguyen Viet Anh	23BA14013	CS

Academic Year: 2025 – 2026

Contents

1 Introduction **2**

2 Background **2**
2.1 Why AES-256-GCM is used? 2
2.2 Where AES-256-GCM is applied? 2

3 Application context **2**
3.1 AES-256-GCM Architecture 2
3.2 Threat model 4

4 Implementation **4**

5 Results and Analysis **6**
5.1 Attack Simulation and Defense Verification 6
5.2 Performance under real banking workloads 6
5.3 Comparative security analysis 6
5.4 The Weakness 7

6 Conclusion **7**

1 Introduction

The rise of electronic banking means that sensitive financial data — PINs, account numbers, transaction amounts — must travel across public networks every time a customer uses internet banking, a mobile application, or an ATM. This creates three fundamental security requirements that any banking cryptographic system must satisfy simultaneously [1]:

- **Confidentiality** — ensuring that transaction data remains unreadable to unauthorized parties
- **Integrity** — ensuring that data cannot be silently altered in transit
- **Authentication** — ensuring that both parties can verify each other's identity before any transaction is processed.

To address all three requirements simultaneously, **AES-256-GCM** (Advanced Encryption Standard, 256-bit key, Galois/Counter Mode) has been widely adopted in modern banking systems — providing data encryption, integrity verification, and identity authentication all in a single pass.

2 Background

2.1 Why AES-256-GCM is used?

AES is a block cipher that requires a mode of operation. Traditional modes such as CBC or CTR provide confidentiality only, AES-256 "standalone" is only a block cipher and does not provide integrity or authentication, while AES-256-GCM is an Authenticated Encryption with Associated Data (AEAD) mode that ensures confidentiality, integrity, and authentication simultaneously [2, 3].

2.2 Where AES-256-GCM is applied?

AES-256-GCM is widely adopted in modern secure communication systems and protocols due to its strong security guarantees and efficiency

- **In Transport Layer Security (TLS): Web & Electronic Banking Security** [4]
  - Example: Online banking transactions (login, money transfer) use HTTPS with cipher suites such as TLS_AES_256_GCM_SHA384.
- **In IPsec (VPN & Network Security)** [5]
  - Example: Enterprise VPN systems (e.g., banking internal networks) use AES-256-GCM to protect sensitive financial data in transit.
- **In Secure Messaging Systems** [6]
  - Example: Secure messaging apps use AES-GCM (or similar AEAD modes) to encrypt messages and ensure integrity.
- **In Cloud Storage & Data Protection** [2]
  - Example: Cloud services encrypt user data using AES-GCM to ensure both confidentiality and integrity.

3 Application context

3.1 AES-256-GCM Architecture

AES-256-GCM combines two operations:

1. **CTR Mode (Counter):** Encrypts plaintext blocks using AES with an incrementing counter, enabling parallel processing.
2. **GHASH (Galois Hash):** Computes a polynomial hash over the ciphertext and any Additional Authenticated Data (AAD) using Galois Field multiplication in $GF(2^{128})$.

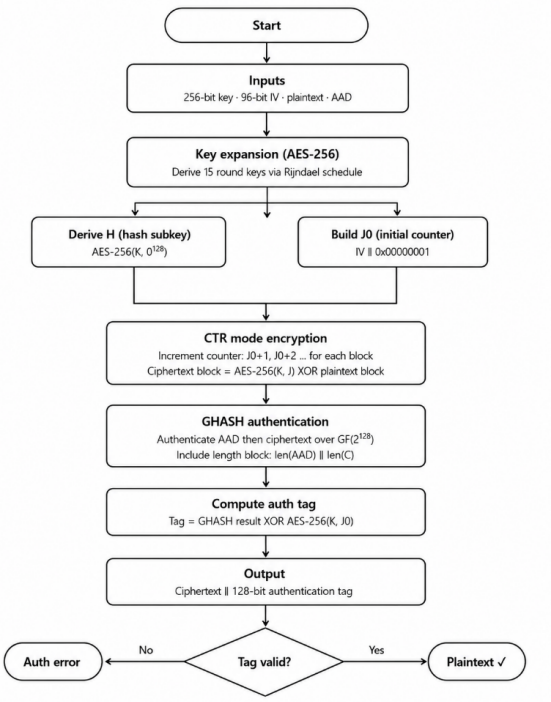


Figure 1: AES-256-GCM Architecture Flow

3.2 Threat model

Application	Security Technology	Mitigated Attacks (Threats Resolved)
1. Internet Banking	TLS 1.3	**Man-in-the-Middle (MitM):** Prevents unauthorized interception or modification of data between the web browser and the server [11]. **Downgrade Attacks:** TLS 1.3 removes legacy cipher suites, preventing attackers from forcing the connection to use older, vulnerable protocols (like SSL 3.0 or TLS 1.0) [12]. **Replay Attacks:** Ensures that intercepted session data cannot be re-used to perform duplicate unauthorized transactions [13].
2. ATM Transaction	PIN Block Encryption	**Network Eavesdropping:** Encrypts the PIN within a specialized block format before it leaves the ATM, ensuring the plain-text PIN is never visible on the internal bank network [14]. **Data Skimming:** Protects against "wire-tapping" at the hardware level where attackers attempt to capture sensitive cardholder data during transmission to the host.
3. Mobile Banking	Certificate Pinning + TLS [15]	**Fraudulent CA/Malicious Certificates:** Blocks attackers from using certificates issued by compromised or rogue Certificate Authorities (CAs) to impersonate the bank's server. **Advanced MitM (Public Wi-Fi):** Even if an attacker controls the network or installs a rogue Root CA on the device, the App will reject the connection if the server's certificate doesn't match the "pinned" identity hardcoded in the App.

Table 1: Security technologies and mitigated attacks in banking applications

4 Implementation

The encrypted packet sent over the network contains five elements [2, 9, 19]:

Component	Description	Role in the transaction
AAD (Associated Authenticated Data)	Cleartext metadata (e.g., "TXNID:2026...|TIME:...")	Not encrypted but integrity-protected; any tampering invalidates the tag. Binds the ciphertext to a specific transaction context.
Ciphertext	Encrypted version of the transfer order	Provides confidentiality — only the holder of the correct key can read it.
Tag (Authentication Tag)	Cryptographic checksum produced during encryption	Guarantees authenticity & integrity of both ciphertext and AAD. Acts as a tamper-evident seal.
Nonce (Number Used Once)	Unique random value for each encryption	Ensures that identical orders produce different ciphertexts; prevents replay attacks.
Salt	Random bytes mixed with the passphrase to derive the key	Makes every transaction key unique even if the same passphrase is reused.

Table 2: Components of the transmitted message

The implementation was successfully realized using the PyCryptodome library.
Core function of client side – Encryption (Sender) [20]

```
# AES-GCM encryption
cipher = AES.new(key, AES.MODE_GCM)
cipher.update(aad.encode())
ciphertext, tag = cipher.encrypt_and_digest(
    sensitive_data.encode()
)
nonce = cipher.nonce
```

Core function of server side – Decryption (Sender) [20]

```
# AES-GCM decryption
cipher = AES.new(
    decryption_key,
    AES.MODE_GCM,
    received_nonce
)
cipher.update(received_aad.encode())
try:
    decrypted_data = cipher.decrypt_and_verify(
        received_ciphertext,
        received_tag
    )
    print("MAC validated - Transaction accepted")
    print("Decrypted order:", decrypted_data.decode())
except ValueError:
    print("MAC validation failed - Transaction rejected")
```

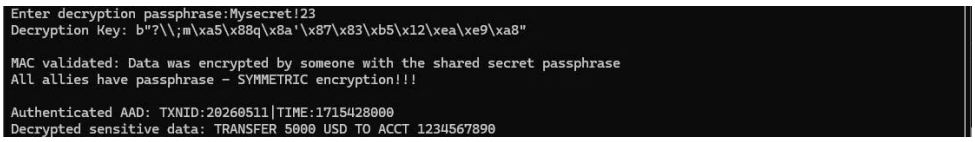


Figure 2: Correct Passphrase

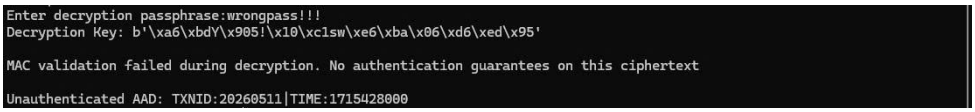


Figure 3: Wrong Passphrase

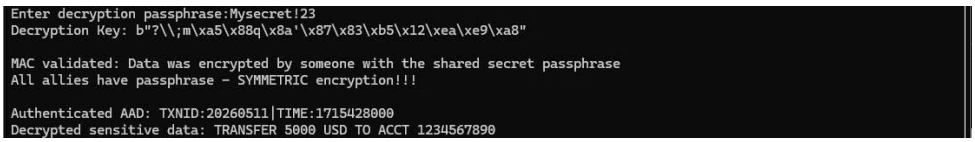


Figure 4: Tampered AAD

5 Results and Analysis

5.1 Attack Simulation and Defense Verification

- **Semantic Security:** Semantic security ensures that encrypting the same plaintext multiple times produces different ciphertexts when different nonces are used [2, 7].
- **Tamper Detection:** AES-GCM provides authenticated encryption, meaning any modification to ciphertext or authentication tag can be detected during decryption [8].
- **Nonce Reuse Vulnerability:** AES-GCM requires a unique nonce for every encryption operation under the same key. Reusing a nonce creates catastrophic vulnerabilities [9].
- **Key Sensitivity:** AES-GCM exhibits high key sensitivity, meaning even a one-bit change in the encryption key produces completely different ciphertext outputs [2].
- **AAD Integrity (Additional Authenticated Data):** AES-GCM supports Additional Authenticated Data (AAD), allowing non-encrypted metadata to be authenticated together with ciphertext [7].

5.2 Performance under real banking workloads

Performance benchmarks across payload sizes further support this:

- **Small payloads ($\leq$ 1 KB):** Slight overhead due to initialization, but still suitable for real-time requests [2].
- **Medium payloads (1–100 KB):** Stable performance for typical transactions (e.g., fund transfer, API calls) [2].
- **Large payloads ($\geq$ 1 MB):** Near-maximum efficiency due to parallel processing and hardware acceleration [2].

Thus, AES-256-GCM maintains consistent and scalable performance across different transaction sizes, aligning with the high-demand requirements of real banking workloads.

5.3 Comparative security analysis

The following table compares AES-256-GCM with other algorithms used in banking environments:

Algorithm	Security Level	Server Throughput	Mobile Throughput	Banking Suitability
AES-256-GCM [2]	256-bit	High (∼4 GB/s)	Moderate (∼2 GB/s)	Optimal (Server/Core)
ChaCha20-Poly1305 [16]	256-bit	Moderate (∼3 GB/s)	High (∼3–4 GB/s)	Optimal (Mobile Apps)
AES-128-GCM [17]	128-bit	Very high (∼5 GB/s)	High (∼3 GB/s)	Lower security margin
3DES [18]	112-bit	Very low (<0.2 GB/s)	Very low	Deprecated only

Table 3: Comparison of symmetric encryption algorithms in banking environments

5.4 The Weakness

- **Nonce Reuse — Critical Vulnerability [10]:** Reusing a nonce under the same key exposes an identical keystream. An attacker with C_1, C_2 computes $C_1 \oplus C_2 = P_1 \oplus P_2$, leaking plaintext — and worse, recovers the GHASH subkey H, allowing arbitrary authenticated forgeries. A real-world Internet scan found 184 HTTPS servers affected, including financial institutions, with a live forgery demonstrated.
- **Truncated Authentication Tag — Forgery Risk [2]:** A 32-bit tag on a 1 KB record gives $\sim 2^{-22}$ success rate — feasible under sustained attack — and each forgery progressively leaks H until authentication collapses entirely.
- **Cache-Timing Side-Channel [2]:** Software AES S-Box lookups vary with key/data bytes, leaking via CPU cache timing. In multi-tenant cloud banking, a co-located attacker can infer key material from shared L3 cache patterns.

6 Conclusion

1. Modern electronic banking exposes sensitive financial data to a fundamentally hostile environment — public networks where any packet can be intercepted, modified, or replayed. AES-256-GCM addresses this reality directly: a single algorithm that encrypts transaction data, detects tampering, and authenticates communication partners, all within the 50µs window that real-time banking demands.
2. No algorithm operates in a vacuum. The nonce reuse vulnerability is not a theoretical concern — it is a deployment mistake that completely undermines every security guarantee GCM provides. And even a perfectly implemented AES-256-GCM cannot protect a transaction that has already been captured in plaintext by malware running on the device.
3. AES-256-GCM is the right tool for the problem it was designed to solve and it solves well. But the assumptions it relies on — unique nonces, secure endpoints, classical adversaries — are worth examining carefully as banking systems scale and quantum computing matures.

References

[1] DestCert, "The Five Pillars of Information Security: CIA Triad and More". Available: https://destcert.com/resources/five-pillars-information-security/

[2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, Nov. 2007. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[3] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST SP 800-38A, Dec. 2001. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf

[4] J. Salowey, A. Choudhury, and D. McGrew, "AES Galois Counter Mode (GCM) Cipher Suites for TLS," RFC 5288, IETF, Aug. 2008. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5288

[5] J. Viega and D. McGrew, "The Use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP)," RFC 4106, IETF, Jun. 2005. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc4106

[6] D. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," NIST Submission, 2004. [Online]. Available: https://eprint.iacr.org/2004/193.pdf

[7] M. Jones and J. Hildebrand, "AES-GCM Authenticated Encryption for SRTP," RFC 7714, IETF, Dec. 2015. [Online]. Available: https://datatracker.ietf.org/doc/rfc7714/

[8] Python Cryptographic Authority, "Authenticated Encryption with Associated Data (AEAD) — Hazmat Primitives," Cryptography.io, v44.0.3. [Online]. Available: https://cryptography.io/en/44.0.3/hazmat/primitives/aead/

[9] D. McGrew, "An Interface and Algorithms for Authenticated Encryption," RFC 5116, IETF, Jan. 2008. [Online]. Available: https://www.ietf.org/rfc/rfc5116.txt

[10] H. Böck, A. Zauner, S. Devlin, J. Somorovsky, and P. Jovanovic, "Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS," *USENIX WOOT*, 2016. [Online]. Available: https://www.usenix.org/system/files/conference/woot16/woot16-paper-bock.pdf

[11] Approov, "How to Prevent MitM Attacks Between Mobile Apps and APIs," [Online]. Available: https://approov.io/how-to-prevent-mitm-attacks-btw-mobile-apps-and-apis

[12] CyberArk, "What is a Downgrade Attack?," [Online]. Available: https://www.cyberark.com/what-is/downgrade-attacks/

[13] Avast, "What is a Replay Attack?," [Online]. Available: https://www.avast.com/c-replay-attack

[14] Fortinet, "What is Eavesdropping?," [Online]. Available: https://www.fortinet.com/resources/cyberglossary/eavesdropping

[15] SSL.com, "What is Certificate Pinning?," [Online]. Available: https://www.ssl.com/blogs/what-is-certificate-pinning

[16] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, IETF, Jun. 2018. [Online]. Available: https://datatracker.ietf.org/doc/rfc8439/

[17] NIST, "Advanced Encryption Standard (AES)," FIPS 197, Updated May 9, 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[18] E. Barker and N. Mouha, "Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher," NIST SP 800-67 Rev. 2, Nov. 2017 (Withdrawn January 1, 2024). [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-67r2.pdf

[19] RSA Laboratories, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, IETF, Sep. 2000. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2898

[20] wolf43, *AES-GCM Example*, GitHub. Available: https://github.com/wolf43/AES-GCM-example/blob/master/aes_gcm.py